



Apresentação do Projeto

Chat Multi- Cliente em Python

Donvicton Monteiro e Wictor Oliveira

Desafio do Projeto

Aplicação em Python com Sockets

O projeto exigiu a criação de um sistema de chat multi-cliente com requisitos técnicos estritos:

- Tecnologia Base: Utilização de sockets primitivos.
- Concorrência: Uso obrigatório de Threads.
- Restrição: Proibição explícita de Websockets.

O objetivo era aplicar na prática e demonstrar um entendimento profundo dos fundamentos da comunicação em rede.

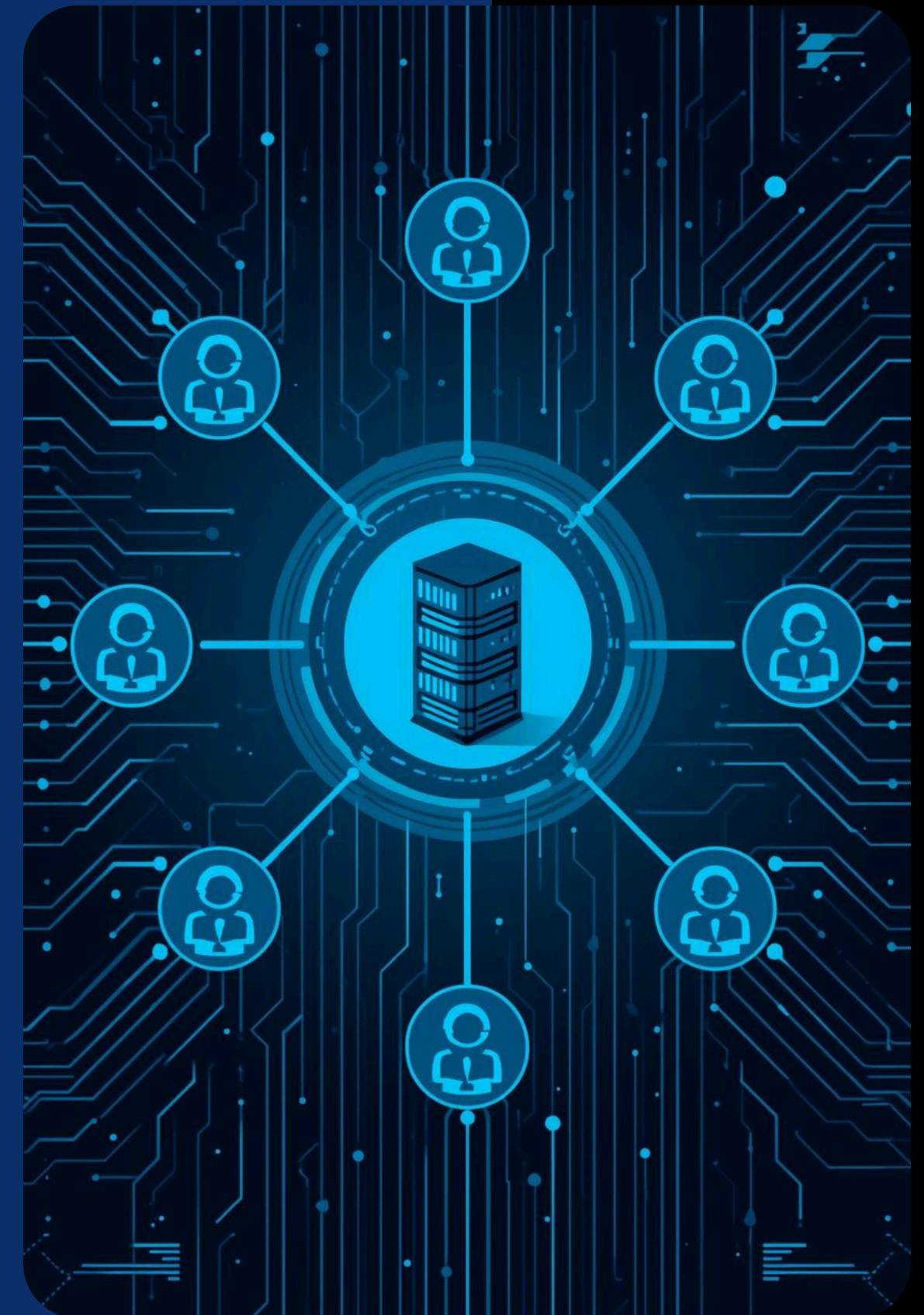


Arquitetura Cliente-Servidor

O modelo centralizado conecta múltiplos clientes simultaneamente.

Funcionamento do Servidor

Escuta e Aceitação de Conexões no Sistema



Threads para Clientes

Gerenciamento de Conexões Simultâneas Eficaz

O Problema: Funções de socket como `accept()` (aceitar um novo cliente) e `recv()` (receber mensagem) são bloqueantes. Isso significa que o servidor "trava" enquanto espera por uma ação. Se não usássemos threads, o servidor só conseguiria atender um cliente de cada vez.

Nossa Solução: Para cada cliente que se conecta, o servidor instancia uma nova Thread dedicada exclusivamente a ele.

Benefícios Imediatos:

- A Thread Principal do servidor fica livre para fazer apenas uma coisa: voltar imediatamente para o `accept()` e esperar por novas conexões.
- Cada Thread de Cliente gerencia a comunicação (recebimento de mensagens, comandos) daquele cliente em paralelo, sem travar o servidor ou os outros clientes.
- Isso nos permite atender dezenas de clientes simultaneamente, cumprindo o requisito obrigatório do projeto.

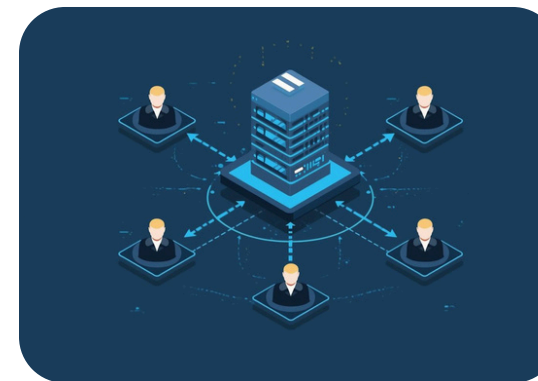
Gerenciamento de Mensagens

Comandos do Servidor



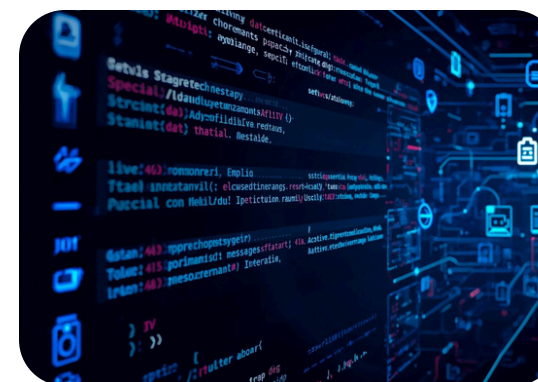
Mensagens em Trânsito

O servidor gerencia mensagens entre clientes, garantindo troca rápida e eficiente de informações essenciais.



Comandos Especiais

O servidor permite comandos especiais, como /msg, para interações específicas entre usuários, melhorando a funcionalidade.



Atividade do Servidor

A interface do servidor exibe a atividade em tempo real, visualizando as interações e comandos dos clientes eficientemente.

Arquitetura do Cliente

Interface do Usuário

Conexão ao Servidor :

```
Digite o IP do Servidor (default 127.0.0.1): 127.0.0.1
Digite seu nickname:
DON
[SYSTEM] Bem-vindo, DON!
[SYSTEM] Digite /list para ver usuários online.
[SYSTEM] Digite /msg <nick> <mensagem> para chat privado.
```

```
|
```

A interface do cliente estabelece uma conexão com o servidor, permitindo troca de mensagens em tempo real.

Arquitetura do Cliente

Interface do Usuário

Comunicação Simultânea :

```
[SYSTEM] Bem-vindo, DON!  
[SYSTEM] Digite /list para ver usuários online.  
[SYSTEM] Digite /msg <nick> <mensagem> para chat privado.
```

```
[SYSTEM] WICTOR entrou no chat.  
[SYSTEM] MARCIO entrou no chat.  
<MARCIO> oi  
<WICTOR> oi  
█
```

O uso de duas threads no cliente melhora a experiência do usuário, permitindo comunicação contínua e sem interrupções.

Arquitetura do Cliente

Interface do Usuário

Chat em Ação :

```
[SYSTEM] Bem-vindo, MARCIO!  
[SYSTEM] Digite /list para ver usuários online.  
[SYSTEM] Digite /msg <nick> <mensagem> para chat privado.  
  
oi  
<WICTOR> oi  
<DON> oi  
[PRIVADO de DON] como vai?  
/msg DONWICTON vou bem.  
[SYSTEM] Erro: Usuário 'DONWICTON' não encontrado.  
/msg DON vou bem.  
[PRIVADO para DON] vou bem.  
<DON> WICTOR choveu hoje em Rio Largo ?  
<WICTOR> Sim, choveu muito!  
□
```

As capturas de tela mostram a funcionalidade do sistema com múltiplos clientes interagindo simultaneamente, evidenciando sua usabilidade.

```
[SYSTEM] Bem-vindo, WICTOR!  
[SYSTEM] Digite /list para ver usuários online.  
[SYSTEM] Digite /msg <nick> <mensagem> para chat privado.  
  
[SYSTEM] MARCIO entrou no chat.  
<MARCIO> oi  
oi  
<DON> oi  
<DON> WICTOR choveu hoje em Rio Largo ?  
Sim, choveu muito!  
□
```

```
[SYSTEM] Bem-vindo, DON!  
[SYSTEM] Digite /list para ver usuários online.  
[SYSTEM] Digite /msg <nick> <mensagem> para chat privado.  
  
[SYSTEM] WICTOR entrou no chat.  
[SYSTEM] MARCIO entrou no chat.  
<MARCIO> oi  
<WICTOR> oi  
oi  
/msg MARCIO como vai?  
[PRIVADO para MARCIO] como vai?  
[PRIVADO de MARCIO] vou bem.  
WICTOR choveu hoje em Rio Largo ?  
<WICTOR> Sim, choveu muito!  
□
```

Conclusão do Projeto

Aprendizados sobre Sockets e Threads

Em conclusão, este projeto foi fundamental para aplicar na prática os conceitos teóricos de Redes de Computadores. Os principais aprendizados foram:

- A construção de um modelo cliente-servidor robusto, capaz de gerenciar a interação de múltiplos usuários.
- O domínio das primitivas de Socket TCP como a principal ferramenta para a comunicação em tempo real.
- O uso de Threads como solução essencial para a concorrência, permitindo ao servidor gerenciar múltiplos clientes simultaneamente.
- A identificação e solução de Condições de Corrida (Race Conditions) com `threading.Lock()`, um passo crucial para garantir a integridade dos dados do servidor.